



Politechnika Krakowska
im. Tadeusza Kościuszki

Bezpieczeństwo Chmur Obliczeniowych

Projekt końcowy:

Lekkie protokoły kryptograficzne oparte na krzywych eliptycznych (ECC) dla
bezpiecznego współdzielenia danych w chmurowych systemach IoT.

Tomasz Gondek
Łukasz Kierzek

Cyberbezpieczeństwo sem. II, CY2

Spis treści

1. Opis problemu	3
2. Cel pracy	3
3. Wykorzystywane Algorytmy	4
3.1. RSA	4
3.2. Algorytmy ECC	4
3.2.1. ECDH P-256	4
3.2.2. X25519	4
4. Opis infrastruktury i urządzeń	4
4.1. Konfiguracja środowiska chmurowego	5
4.1.1. Specyfikacja serwera	5
4.1.2. Konfiguracja sieci	5
4.1.3. FastAPI	5
4.1.3.1. Endpoint /ping	6
4.1.3.2. Endpoint /benchmark	6
4.1.3.3. Endpoint /ecdh/p256	6
4.1.3.4. Endpoint /ecdh/x25519	6
4.2. Urządzenie IoT	6
5. Testy	7
5.1. Generowanie kluczy	7
5.1.1. ESP32	7
5.1.2. Chmura	8
5.2. Generowanie wspólnego sekretu	9
5.2.1. ESP32	9
5.2.2. Chmura	10
5.3. Pełny handshake	11
5.3.1. ECDH P-256	12
5.3.2. X25519	13
5.4. Porównanie wyników	13
6. Wnioski Końcowe	14

1. Opis problemu

Rozwój Internetu Rzeczy (IoT) doprowadził do powstania rozproszonych systemów składających się z dużej liczby urządzeń komunikujących się z infrastrukturą sieciową oraz usługami chmurowymi. Urządzenia te zbierają, przetwarzają i przesyłają dane wykorzystywane między innymi w inteligentnych budynkach, systemach przemysłowych, monitoringu środowiskowym czy opiece zdrowotnej. Wraz ze wzrostem liczby połączonych urządzeń rośnie znaczenie zapewnienia odpowiedniego poziomu bezpieczeństwa przesyłanych informacji.

Komunikacja pomiędzy urządzeniami IoT a platformami chmurowymi odbywa się najczęściej za pośrednictwem publicznych sieci teleinformatycznych, co naraża przesyłane dane na różnego rodzaju zagrożenia. Do najczęstszych należą podsłuchiwanie transmisji, modyfikacja przesyłanych informacji, podszywanie się pod uprawnione urządzenia oraz przechwycenie kluczy kryptograficznych wykorzystywanych do zabezpieczania komunikacji. Z tego względu niezbędne jest stosowanie skutecznych mechanizmów kryptograficznych gwarantujących poufność, integralność oraz autentyczność danych.

Istotnym wyzwaniem w projektowaniu bezpiecznych systemów IoT są jednak ograniczone zasoby urządzeń końcowych. W przeciwieństwie do komputerów osobistych lub serwerów, urządzenia IoT dysponują niewielką mocą obliczeniową, ograniczoną pamięcią operacyjną oraz często są zasilane bateryjnie. Powoduje to, że stosowanie tradycyjnych algorytmów kryptograficznych może prowadzić do zwiększonego zużycia energii, wydłużenia czasu komunikacji oraz obniżenia wydajności całego systemu.

Jednym z kluczowych elementów bezpiecznej komunikacji jest proces wymiany kluczy kryptograficznych wykorzystywanych następnie przez szyfry symetryczne do ochrony przesyłanych danych. Przez wiele lat szeroko stosowanym rozwiązaniem była kryptografia RSA, jednak rosnące wymagania dotyczące bezpieczeństwa wymuszają stosowanie coraz większych kluczy, co przekłada się na wzrost kosztów obliczeniowych. Alternatywę stanowią algorytmy oparte na kryptografii krzywych eliptycznych (ECC), które umożliwiają osiągnięcie porównywalnego poziomu bezpieczeństwa przy znacznie mniejszych rozmiarach kluczy oraz niższym zapotrzebowaniu na zasoby sprzętowe.

Szczególne znaczenie w nowoczesnych systemach komunikacyjnych zyskały mechanizmy wymiany kluczy wykorzystujące kryptografię krzywych eliptycznych, takie jak ECDH z krzywą P-256 oraz X25519. Rozwiązania te są obecnie stosowane między innymi w protokołach TLS oraz systemach komunikacji wykorzystujących architekturę chmurową. Ich właściwości sprawiają, że są one postrzegane jako szczególnie atrakcyjne dla środowisk IoT, w których istotne znaczenie mają wydajność, zużycie energii oraz bezpieczeństwo transmisji.

2. Cel pracy

Celem pracy jest analiza oraz ocena efektywności wybranych protokołów kryptograficznych wykorzystywanych do bezpiecznego współdzielenia danych w chmurowych systemach Internetu Rzeczy. Uwagę poświęcono lekkim rozwiązaniom opartym na kryptografii krzywych eliptycznych, tj. ECDH P-256 oraz X25519, które porównano z tradycyjnym podejściem wykorzystującym algorytm RSA-2048.

W ramach części badawczej opracowano środowisko testowe składające się z urządzenia IoT ESP32 oraz serwera działającego w środowisku chmurowym Microsoft Azure. Przeprowadzone eksperymenty obejmują pomiar czasu generowania kluczy, czasu ustanawiania bezpiecznej sesji komunikacyjnej, narzutu komunikacyjnego oraz wykorzystania zasobów urządzenia..

3. Wykorzystywane Algorytmy

3.1. RSA

RSA jest algorytmem kryptografii asymetrycznej wykorzystującym parę kluczy: publiczny i prywatny. Jego bezpieczeństwo opiera się na trudności faktoryzacji dużych liczb całkowitych będących iloczynem dwóch liczb pierwszych.

Algorytm znajduje zastosowanie w szyfrowaniu danych, podpisach cyfrowych oraz wymianie kluczy. Poziom bezpieczeństwa RSA zależy od długości klucza, która obecnie wynosi najczęściej 2048 lub 3072 bity.

3.2. Algorytmy ECC

ECC jest grupą algorytmów asymetrycznych wykorzystujących własności matematyczne krzywych eliptycznych nad ciałami skończonymi. Bezpieczeństwo ECC opiera się na trudności rozwiązania problemu logarytmu dyskretnego na krzywych eliptycznych.

W porównaniu z RSA, algorytmy ECC zapewniają porównywalny poziom bezpieczeństwa przy znacznie mniejszych długościach kluczy. Przekłada się to na mniejsze zużycie pamięci, krótszy czas wykonywania operacji kryptograficznych oraz niższe zapotrzebowanie na energię, co czyni je szczególnie atrakcyjnymi dla urządzeń Internetu Rzeczy.

3.2.1. ECDH P-256

ECDH (Elliptic Curve Diffie-Hellman) jest protokołem wymiany kluczy wykorzystującym kryptografię krzywych eliptycznych. W niniejszej pracy wykorzystano krzywą P-256, należącą do zestawu standardowych krzywych rekomendowanych przez NIST.

Protokół umożliwia dwóm stronom uzgodnienie wspólnego sekretu bez konieczności przesyłania go przez sieć. Uzyskany klucz może następnie zostać wykorzystany przez algorytmy szyfrowania symetrycznego do zabezpieczenia komunikacji.

3.2.2. X25519

X25519 jest implementacją protokołu Diffiego-Hellmana opartą na krzywej Curve25519. Został zaprojektowany z myślą o wysokiej wydajności, prostocie implementacji oraz odporności na wybrane ataki wynikające z błędów implementacyjnych.

Algorytm wykorzystuje 256-bitowe klucze i jest obecnie szeroko stosowany w nowoczesnych protokołach bezpieczeństwa, takich jak TLS 1.3. Dzięki niewielkim wymaganiom obliczeniowym oraz wysokiemu poziomowi bezpieczeństwa X25519 jest często uznawany za jedno z najbardziej efektywnych rozwiązań do wymiany kluczy w systemach IoT.

4. Opis infrastruktury i urządzeń

W celu przeprowadzenia badań przygotowano środowisko testowe składające się z urządzenia Internetu Rzeczy oraz serwera działającego w chmurze obliczeniowej. Architektura została zaprojektowana w sposób odzwierciedlający rzeczywisty scenariusz komunikacji pomiędzy urządzeniem IoT a usługą chmurową.

Urządzeniem klienckim jest moduł ESP32-CAM wyposażony w mikrokontroler ESP32 oraz interfejs Wi-Fi. Moduł odpowiada za generowanie danych testowych, realizację operacji kryptograficznych oraz komunikację z serwerem. Dzięki ograniczonym zasobom obliczeniowym i pamięciowym ESP32 stanowi reprezentatywną platformę dla współczesnych urządzeń IoT.

Po stronie chmurowej wykorzystano maszynę wirtualną działającą w środowisku Microsoft Azure. Na serwerze uruchomiona została aplikacja implementująca badane mechanizmy kryptograficzne oraz obsługująca komunikację z urządzeniem IoT. Serwer odpowiada za wymianę kluczy kryptograficznych, ustanowienie bezpiecznej sesji oraz odbiór danych przesyłanych przez urządzenie.

Komunikacja pomiędzy urządzeniem IoT a serwerem realizowana jest za pośrednictwem sieci Wi-Fi oraz protokołu HTTP. W trakcie badań analizowane są dwa mechanizmy ustanawiania bezpiecznego kanału komunikacyjnego: ECDH P-256 oraz X25519. W celu oceny wydajności poszczególnych rozwiązań rejestrowane są czasy generowania kluczy kryptograficznych oraz czasy ustanawiania bezpiecznej sesji

4.1. Konfiguracja środowiska chmurowego

W celu realizacji badań utworzono środowisko serwerowe w chmurze Microsoft Azure. Wykorzystano maszynę wirtualną z systemem Ubuntu Server 24.04 LTS, pełniącą rolę serwera komunikacyjnego dla urządzenia IoT.

4.1.1. Specyfikacja serwera

Parametr	Wartość
Nazwa maszyny	iot-crypto-vm
Dostawca chmury	Microsoft Azure
System operacyjny	Ubuntu Server 24.04 LTS
Architektura	x86-64
Typ maszyny	Standard B2ls v2
Liczba vCPU	2
Pamięć RAM	4 GiB
Publiczny adres IP	Tak
Prywatny adres IP	10.0.0.4

Tabela 1: Specyfikacja maszyny wirtualnej wykorzystanej w środowisku badawczym.

4.1.2. Konfiguracja sieci

W celu umożliwienia komunikacji między urządzeniem IoT a chmurą, została dodana zasada pozwalająca na ruch na porcie tcp/8000 oraz został przypisany publiczny adres IP.

350	FastAPI	8000	TCP	Dowolne	Dowolne	Allow
-----	---------	------	-----	---------	---------	-------

Rysunek 1: Zasada sieciowa tcp/8000 allow

4.1.3. FastAPI

kod znajduje się w pliku main_benchmarks.py

W celu realizacji komunikacji pomiędzy urządzeniem IoT a środowiskiem chmurowym opracowano aplikację serwerową wykorzystującą framework FastAPI. Serwer został uruchomiony na maszynie wirtualnej Azure i udostępniał zestaw endpointów wykorzystywanych do testów wydajnościowych oraz wymiany kluczy kryptograficznych.

4.1.3.1. Endpoint /ping

Endpoint testowy wykorzystywany do weryfikacji dostępności serwera oraz poprawności połączenia sieciowego pomiędzy urządzeniem ESP32-CAM a maszyną wirtualną. Po otrzymaniu żądania serwer zwracał prostą odpowiedź zawierającą komunikat „pong”.

4.1.3.2. Endpoint /benchmark

Endpoint służący do zapisywania wyników pomiarów wykonywanych przez urządzenie IoT. Po zakończeniu benchmarku ESP32-CAM przesyłało do serwera informacje o zastosowanym algorytmie, rodzaju operacji, liczbie iteracji oraz wartościach minimalnych, średnich i maksymalnych. Otrzymane dane były automatycznie zapisywane do pliku CSV znajdującego się na serwerze, co umożliwiało późniejszą analizę wyników.

4.1.3.3. Endpoint /ecdh/p256

Endpoint realizujący wymianę kluczy przy użyciu algorytmu ECDH opartego na krzywej eliptycznej P-256. Urządzenie ESP32-CAM przesyłało do serwera swój klucz publiczny. Następnie serwer generował własną parę kluczy, obliczał wspólny sekret oraz odsyłał swój klucz publiczny. Po odebraniu odpowiedzi urządzenie wyznaczało identyczny wspólny sekret po swojej stronie.

Endpoint ten był wykorzystywany podczas benchmarków pełnego procesu ustanawiania bezpiecznej sesji z wykorzystaniem standardowej krzywej NIST P-256.

4.1.3.4. Endpoint /ecdh/x25519

Endpoint odpowiedzialny za wymianę kluczy z wykorzystaniem algorytmu X25519 bazującego na krzywej Curve25519. Mechanizm działania był analogiczny do endpointu /ecdh/p256. Po odebraniu klucza publicznego urządzenia serwer generował własną parę kluczy X25519, obliczał wspólny sekret i zwracał swój klucz publiczny.

Endpoint ten umożliwił przeprowadzenie porównania wydajności nowoczesnego algorytmu X25519 z klasycznym rozwiązaniem ECDH P-256 w warunkach rzeczywistej komunikacji pomiędzy urządzeniem IoT a chmurą obliczeniową.

4.2. Urządzenie IoT

Jako urządzenie IoT wykorzystano moduł ESP32 na płytce ESP32-CAM. Mikrokontroler posiada wbudowaną komunikację Wi-Fi oraz wystarczającą moc obliczeniową do realizacji nowoczesnych algorytmów kryptograficznych przy zachowaniu niskiego zużycia energii.

ESP32 został wybrany ze względu na szerokie zastosowanie w systemach Internetu Rzeczy oraz wsparcie biblioteki mbedTLS, umożliwiającą implementację kryptografii opartej na ECC.

W projekcie moduł pełni rolę urządzenia końcowego komunikującego się z serwerem za pośrednictwem sieci Wi-Fi. Jego zadaniem jest generowanie kluczy kryptograficznych oraz bezpieczne przesyłanie danych do środowiska chmurowego.



Rysunek 2: ESP32-CAM

5. Testy

5.1. Generowanie kluczy

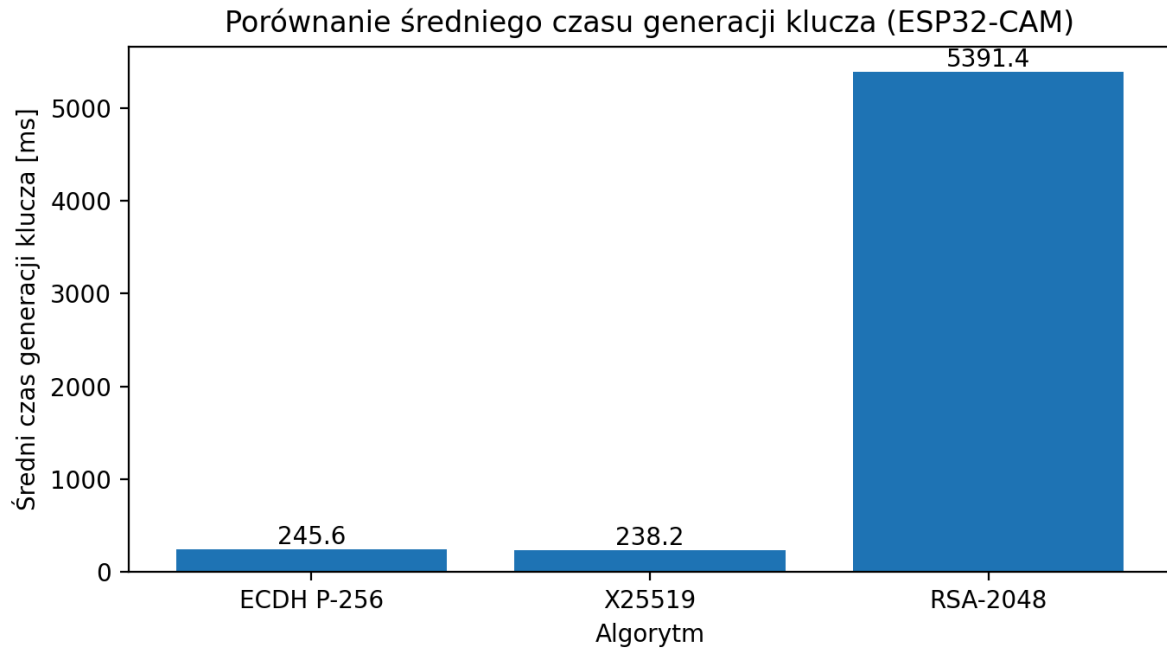
5.1.1. ESP32

kod benchmarka znajduje się w pliku keygen_benchmark.cpp

W tym benchmarku został przetestowany czas generowania 20 kluczy przy użyciu biblioteki wbudowanej w ESP IDF.

Algorytm	Min [ms]	Średnia [ms]	Max [ms]	Iteracje
ECDH P-256	245.1	245.6	246.4	20
X25519	236.5	238.2	239.7	20
RSA-2048	735.2	5391.4	17772.0	20

Tabela 2: Wyniki benchmarku generacji kluczy na platformie ESP32-CAM.



Rysunek 3: Porównanie czasów na ESP32

Uzyskane wyniki pokazują wyraźną przewagę algorytmów opartych na kryptografii krzywych eliptycznych nad RSA-2048. Średni czas generacji klucza dla X25519 wyniósł 238,2 ms, a dla ECDH P-256 245,6 ms, co oznacza bardzo zbliżoną wydajność obu rozwiązań. Najszybszy okazał się algorytm X25519.

Znacznie gorsze wyniki uzyskano dla RSA-2048, którego średni czas generacji klucza wyniósł 5391,4 ms (5,39 s). Oznacza to, że generacja klucza RSA była ponad 22 razy wolniejsza niż w przypadku algorytmów ECC.

Dodatkowo ECDH P-256 i X25519 charakteryzowały się bardzo małym rozrzutem wyników, co świadczy o stabilnym czasie wykonywania operacji. W przypadku RSA-2048 zaobserwowano dużą zmienność czasów (od 735 ms do 17 772 ms), wynikającą z procesu generowania liczb pierwszych.

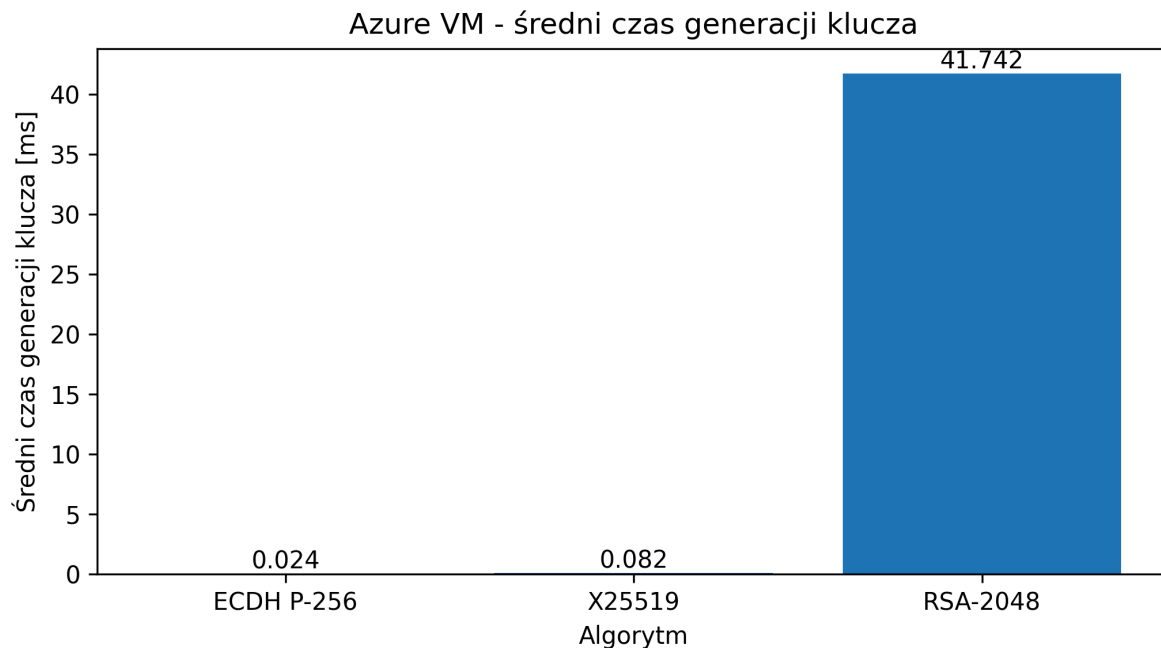
Wyniki potwierdzają, że algorytmy ECC, szczególnie X25519, są znacznie lepiej dostosowane do zastosowań w urządzeniach IoT o ograniczonych zasobach obliczeniowych.

5.1.2. Chmura

kod benchmarka znajduje się w pliku `keygen_benchmark_vm.py`

Algorytm	Min [ms]	Średnia [ms]	Max [ms]	Iteracje
ECDH P-256	0.014	0.024	0.794	100
X25519	0.033	0.082	4.814	100
RSA-2048	9.446	41.742	118.792	100

Tabela 3: Wyniki benchmarku generacji kluczy na maszynie wirtualnej Azure.



Rysunek 4: Porównanie czasów na Azure VM

Na podstawie wyników uzyskanych na maszynie wirtualnej Azure można stwierdzić, że koszt obliczeniowy generacji kluczy jest dla infrastruktury chmurowej praktycznie pomijalny. Wszystkie badane algorytmy osiągnęły czasy wykonania rzędu mikro- lub pojedynczych milisekund.

Najkrótszy średni czas generacji klucza uzyskał algorytm ECDH P-256 (0,024 ms), natomiast X25519 osiągnął wynik 0,082 ms. Różnice pomiędzy tymi algorytmami są niewielkie i w praktyce nie mają istotnego wpływu na wydajność systemu działającego w chmurze.

Znacznie wolniejszy okazał się RSA-2048, którego średni czas generacji klucza wyniósł 41,742 ms. Mimo że jest to wartość wielokrotnie większa niż dla algorytmów ECC, nadal pozostaje ona bardzo mała z punktu widzenia serwera chmurowego.

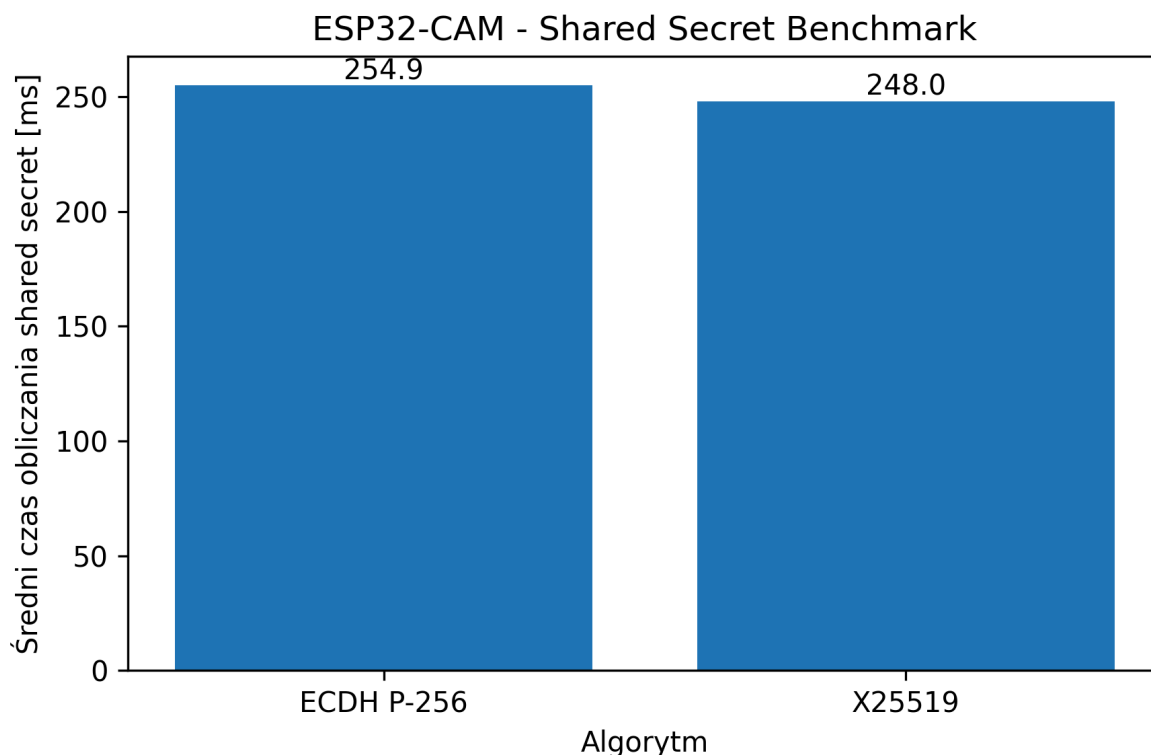
5.2. Generowanie wspólnego sekretu

5.2.1. ESP32

kod benchmarka znajduje się w pliku `shared_secret.cpp`

Algorytm	Min [ms]	Średnia [ms]	Max [ms]	Iteracje
ECDH P-256	253.2	254.9	256.2	100
X25519	244.4	248.0	251.6	100

Tabela 4: Wyniki benchmarku obliczania wspólnego sekretu na platformie ESP32-CAM.



Rysunek 5: Czasy obliczania shared secret

Uzyskane wyniki pokazują, że oba badane algorytmy kryptografii krzywych eliptycznych osiągają bardzo zbliżoną wydajność podczas obliczania wspólnego sekretu na platformie ESP32-CAM. Średni czas wykonania operacji wyniósł 254,9 ms dla ECDH P-256 oraz 248,0 ms dla X25519.

Spośród analizowanych rozwiązań nieznacznie lepszy wynik uzyskał algorytm X25519, który był około 2,7% szybszy od ECDH P-256. Różnica ta jest jednak niewielka i w praktyce oba algorytmy oferują porównywalną wydajność.

Dodatkowo dla obu algorytmów zaobserwowano bardzo mały rozrzut wyników pomiędzy kolejnymi pomiarami. Świadczy to o stabilnym i przewidywalnym czasie wykonywania operacji, co jest pożądaną cechą w systemach IoT działających w czasie rzeczywistym.

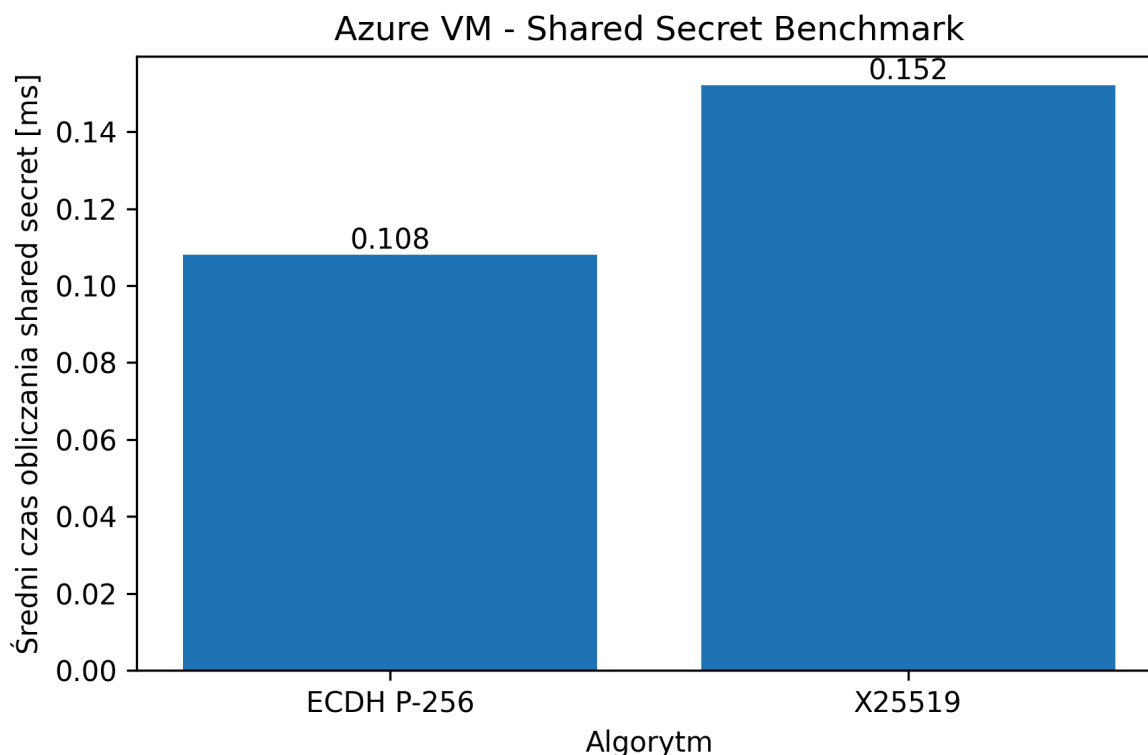
Porównując wyniki z wcześniejszym benchmarkiem generacji kluczy można zauważyć, że czas obliczania wspólnego sekretu jest zbliżony do czasu generacji pary kluczy. Oznacza to, że oba etapy mają podobny udział w całkowitym czasie ustanawiania bezpiecznej sesji komunikacyjnej.

5.2.2. Chmura

kod benchmarka znajduje się w pliku `benchmark_shared_secrets.py`

Algorytm	Min [ms]	Średnia [ms]	Max [ms]	Iteracje
ECDH P-256	0.095	0.108	1.032	100
X25519	0.103	0.152	4.864	100

Tabela 5: Wyniki benchmarku obliczania wspólnego sekretu na maszynie wirtualnej Azure.



Rysunek 6: Czasy obliczania shared secret na Azure

Uzyskane wyniki pokazują, że obliczanie wspólnego sekretu na maszynie wirtualnej Azure jest bardzo szybkie i zajmuje ułamki milisekundy. Średni czas wykonania operacji wyniósł 0,108 ms dla ECDH P-256 oraz 0,152 ms dla X25519.

W przeciwieństwie do wyników uzyskanych na ESP32-CAM, nieco lepszą wydajność osiągnął algorytm ECDH P-256. Różnica pomiędzy badanymi algorytmami wyniosła około 0,044 ms, dlatego z punktu widzenia infrastruktury chmurowej można uznać ją za nieistotną.

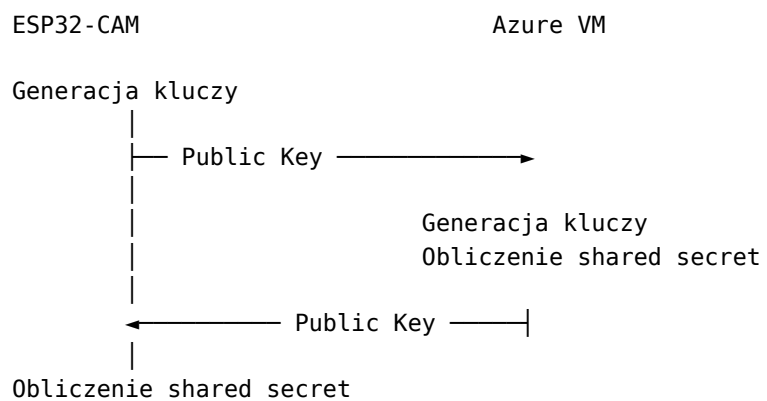
Porównanie z wynikami uzyskanymi na urządzeniu IoT pokazuje ogromną przewagę wydajnościową środowiska chmurowego. Operacja obliczania wspólnego sekretu była około 2360 razy szybsza dla ECDH P-256 oraz około 1630 razy szybsza dla X25519 niż na ESP32-CAM.

5.3. Pełny handshake

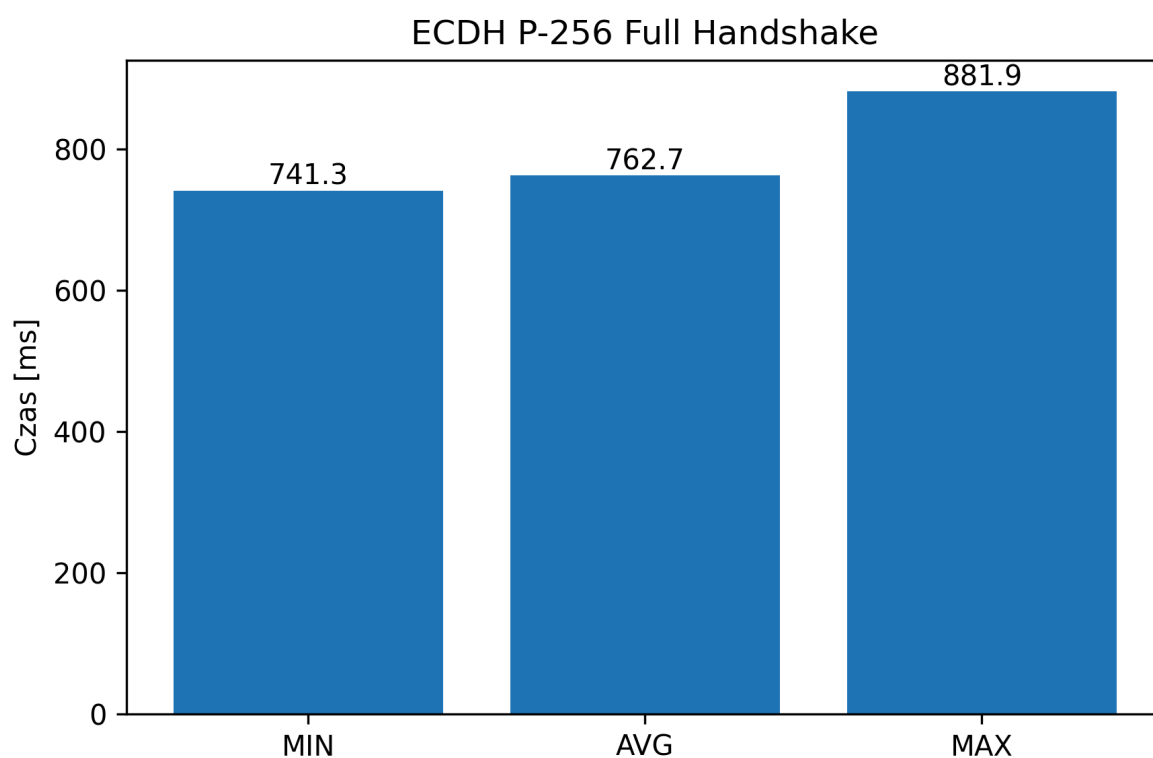
kod benchmarka znajduje się w pliku `benchmark_full_handshake.cpp` oraz `benchmark_full.py`

W trakcie pojedynczej iteracji urządzenie ESP32-CAM generowało parę kluczy dla badanego algorytmu (ECDH P-256 lub X25519), a następnie wysłało swój klucz publiczny do serwera FastAPI uruchomionego na maszynie wirtualnej Azure. Po odebraniu żądania serwer generował własną parę kluczy, obliczał wspólny sekret oraz odsyłał klucz publiczny do urządzenia IoT.

Po otrzymaniu odpowiedzi ESP32-CAM importowało klucz publiczny serwera i wyznaczało wspólny sekret przy użyciu odpowiedniego algorytmu wymiany kluczy. Czas pomiaru obejmował cały proces: generację kluczy, komunikację sieciową HTTP, przetwarzanie danych JSON oraz obliczenie wspólnego sekretu.

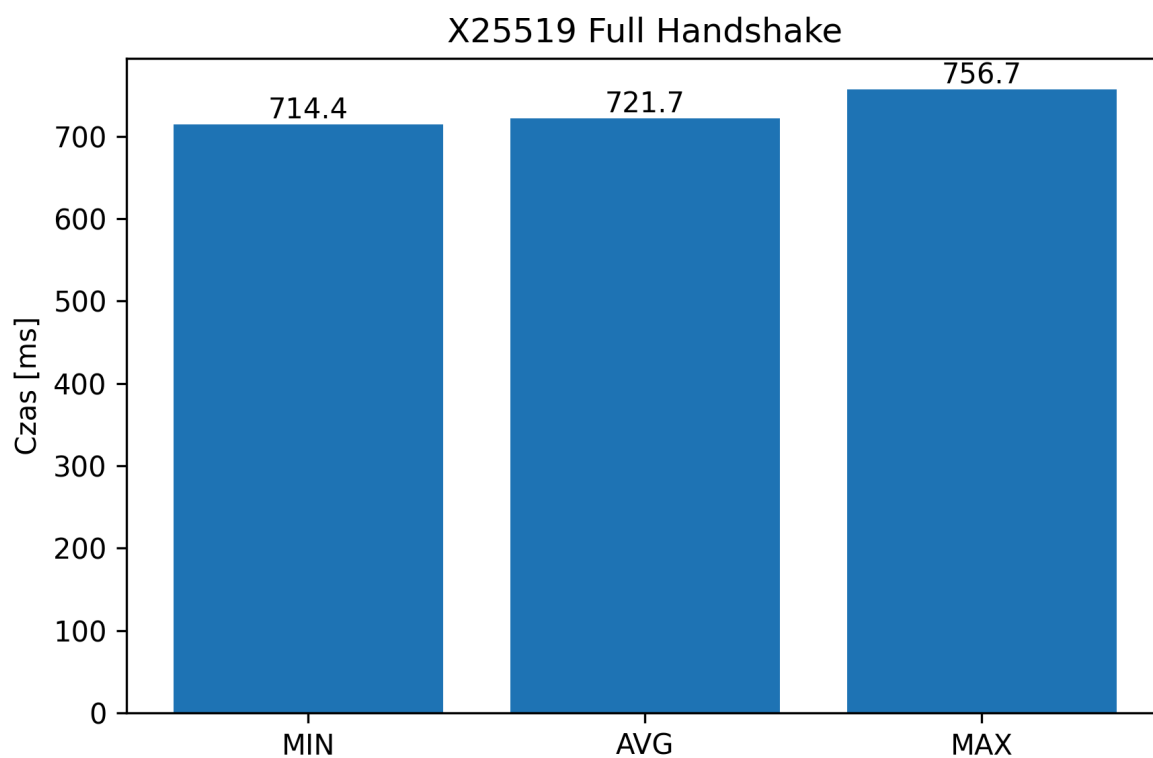


5.3.1. ECDH P-256



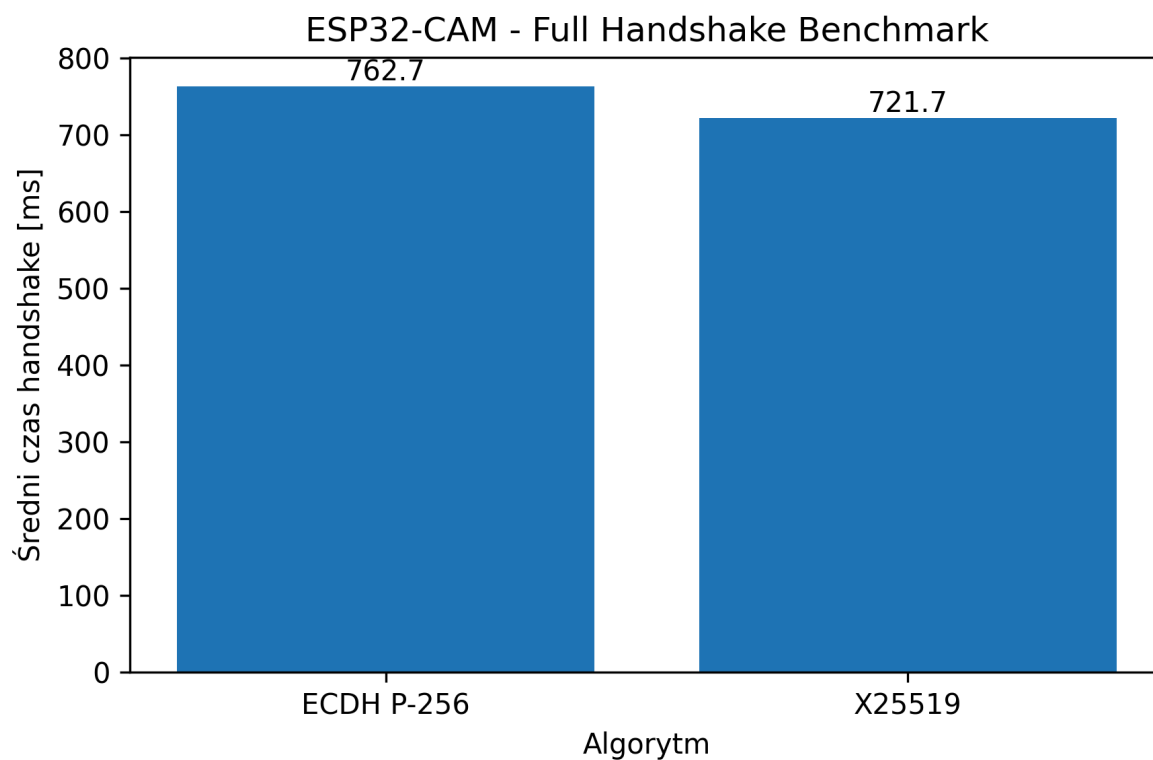
Rysunek 7: Czasy ustanowienia bezpiecznej sesji dla ECDH

5.3.2. X25519



Rysunek 8: Czasy ustanowienia bezpiecznej sesji dla X25519

5.4. Porównanie wyników



Rysunek 9: Porównanie ECDH i X25519

6. Wnioski Końcowe

Przeprowadzone pomiary wykazały znaczącą przewagę rozwiązań opartych na kryptografii krzywych eliptycznych nad algorytmem RSA. W przypadku generacji kluczy na urządzeniu ESP32-CAM algorytmy ECDH P-256 oraz X25519 wymagały około 240 ms, podczas gdy generacja klucza RSA-2048 trwała średnio ponad 5 sekund. Oznacza to, że zastosowanie RSA na urządzeniach IoT o ograniczonych zasobach jest znacznie mniej efektywne niż wykorzystanie algorytmów ECC.

W testach obliczania wspólnego sekretu algorytmy ECDH P-256 oraz X25519 osiągnęły zbliżone wyniki zarówno na ESP32-CAM, jak i na maszynie wirtualnej Azure. Jednocześnie operacje wykonywane w środowisku chmurowym były ponad tysiącrotnie szybsze niż na urządzeniu IoT, co potwierdza, że główne ograniczenia wydajnościowe występują po stronie urządzeń końcowych.

Najbardziej istotnym eksperymentem był benchmark pełnego procesu ustanawiania bezpiecznej sesji pomiędzy urządzeniem IoT a chmurą. Uzyskane wyniki wykazały, że średni czas pełnego handshaku wyniósł około 763 ms dla ECDH P-256 oraz 722 ms dla X25519. Oznacza to, że algorytm X25519 był około 5% szybszy od ECDH P-256, a dodatkowo charakteryzował się mniejszym rozrzutem wyników.

Uzyskane rezultaty potwierdzają, że zastosowanie lekkich protokołów opartych na ECC pozwala znacząco ograniczyć koszty obliczeniowe po stronie urządzeń IoT, co czyni je szczególnie atrakcyjnymi w środowiskach o ograniczonych zasobach sprzętowych oraz energetycznych.

Wykaz rysunków

Rysunek 1 Zasada sieciowa tcp/8000 allow	5
Rysunek 2 ESP32-CAM	7
Rysunek 3 Porównanie czasów na ESP32	8
Rysunek 4 Porównanie czasów na Azure VM	9
Rysunek 5 Czasy obliczania shared secret	10
Rysunek 6 Czasy obliczania shared secret na Azure	11
Rysunek 7 Czasy ustanowienia bezpiecznej sesji dla ECDH	12
Rysunek 8 Czasy ustanowienia bezpiecznej sesji dla X25519	13
Rysunek 9 Porównanie ECDH i X25519	13

Wykaz tabel

Tabela 1	Specyfikacja maszyny wirtualnej wykorzystanej w środowisku badawczym.	5
Tabela 2	Wyniki benchmarku generacji kluczy na platformie ESP32-CAM.	7
Tabela 3	Wyniki benchmarku generacji kluczy na maszynie wirtualnej Azure.	8
Tabela 4	Wyniki benchmarku obliczania wspólnego sekretu na platformie ESP32-CAM.	9
Tabela 5	Wyniki benchmarku obliczania wspólnego sekretu na maszynie wirtualnej Azure.	10